

# Aproximação de $\pi$ usando Série Matemática

Análise de Precisão e Desempenho Computacional

Eugênio Vitor Lopes dos Santos

DCA3703 – Programação Paralela, Universidade Federal do Rio Grande do Norte

2026

## 1 Enunciado

Implementar em C um programa para calcular a aproximação de  $\pi$  por meio de uma série matemática, variando o número de iterações e mensurando o tempo de execução. O objetivo envolve comparar os valores obtidos com a constante de referência  $\pi$ , analisar a evolução da acurácia com o aumento do processamento e avaliar como esse comportamento se manifesta em aplicações reais de alto desempenho, tais como simulações físicas e inteligência artificial.

## 2 Fundamentação Teórica

### 2.1 Série de Leibniz

A série de Leibniz para  $\pi$  é dada por:

$$\pi = 4 \sum_{k=0}^{\infty} \frac{(-1)^k}{2k+1} = 4 \left( 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \dots \right)$$

### 2.2 Algoritmo

1. Inicializar `pi` com 0.0;
2. Para cada iteração  $k$  de 0 até  $N - 1$ :
  - Calcular o termo:  $\frac{1}{2k+1}$ ;
  - Aplicar sinal alternado: positivo para  $k$  par, negativo para  $k$  ímpar;
  - Acumular o termo no somatório;
3. Multiplicar o resultado por 4;
4. Calcular o erro absoluto:  $|\pi_{\text{aprox}} - \pi_{\text{real}}|$ .

## 3 Experimento

### 3.1 Código-Fonte

O programa recebe o número de iterações como argumento de linha de comando e calcula a aproximação de  $\pi$  pela série de Leibniz.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <math.h>
4
5 int main(int argc, char *argv[]) {
6     if (argc != 2) {
7         printf("Uso: %s <numero_de_iteracoes>\n", argv[0]);
8         return 1;
9     }
10
11     long long N = atoll(argv[1]);
12     double pi = 0.0;
13
14     for (long long k = 0; k < N; k++) {
15         double termo = 1.0 / (2.0 * k + 1.0);
16         pi += (k % 2 == 0) ? termo : -termo;
17     }
18     pi *= 4.0;
19
20     double erro = fabs(pi - M_PI);
21
22     printf("Iteracoes: %lld\n", N);
23     printf("Pi: %.15f\n", pi);
24     printf("Erro: %.2e\n", erro);
25
26     return 0;
27 }
```

**Listing 1:** Código-fonte em C para aproximação de  $\pi$

O tempo de execução é medido externamente por meio do utilitário `time` do shell Linux, variando-se o argumento de iterações a cada execução:

```
1 gcc tarefa01.c -o tarefa01 -lm
2 time ./tarefa01 1000000000
```

**Listing 2:** Exemplo de execução com medição de tempo

## 4 Resultados

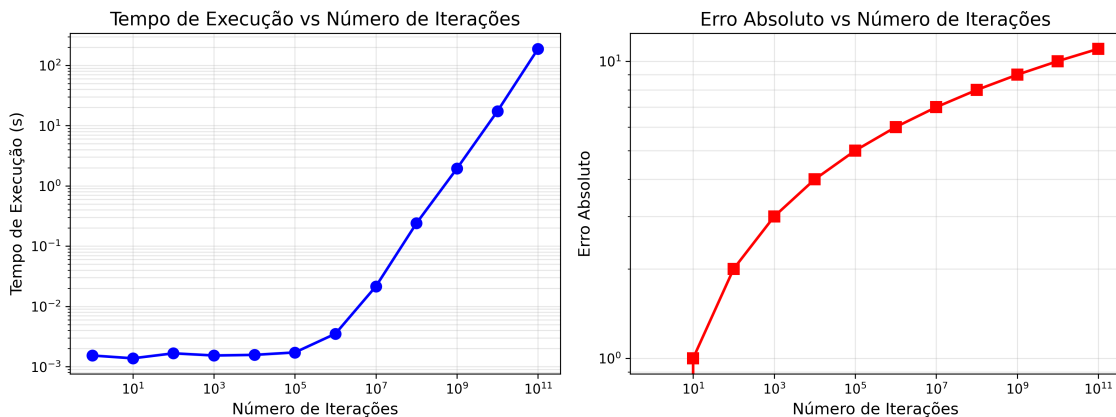
### 4.1 Dados Coletados

**Table 1:** Resultados da aproximação de  $\pi$  em função do número de iterações.

| Iterações       | $\pi$ Aproximado  | Erro Absoluto | Tempo (s) |
|-----------------|-------------------|---------------|-----------|
| 1               | 4.000000000000000 | 8.58e-01      | 0.002     |
| 10              | 3.041839618929403 | 9.98e-02      | 0.001     |
| 100             | 3.131592903558554 | 1.00e-02      | 0.002     |
| 1.000           | 3.140592653839794 | 1.00e-03      | 0.002     |
| 10.000          | 3.141492653590034 | 1.00e-04      | 0.002     |
| 100.000         | 3.141582653589720 | 1.00e-05      | 0.002     |
| 1.000.000       | 3.141591653589774 | 1.00e-06      | 0.004     |
| 10.000.000      | 3.141592553589792 | 1.00e-07      | 0.022     |
| 100.000.000     | 3.141592643589326 | 1.00e-08      | 0.240     |
| 1.000.000.000   | 3.141592652588050 | 1.00e-09      | 1.943     |
| 10.000.000.000  | 3.141592653488346 | 1.01e-10      | 17.408    |
| 100.000.000.000 | 3.141592653578537 | 1.13e-11      | 188.135   |

### 4.2 Análise Visual

Análise de Aproximação de  $\pi$  com Série de Leibniz



**Figure 1:** Análise da aproximação de  $\pi$ : (a) Tempo de execução vs iterações; (b) Erro absoluto vs iterações. Ambos os eixos em escala logarítmica.

### 4.3 Observações Principais

- O erro absoluto decresce em uma ordem de magnitude a cada aumento de 10 vezes no número de iterações;
- O tempo de execução cresce linearmente em relação ao número de iterações processadas;
- Atinge-se precisão da ordem de  $10^{-9}$  com  $10^9$  iterações.

## 5 Discussão

A relação de compromisso (*trade-off*) entre precisão numérica e custo computacional motiva o emprego de técnicas de paralelização e de séries com taxas de convergência superiores. Métodos alternativos à série de Leibniz, tais como a fórmula de Machin ou os algoritmos baseados em Ramanujan e Chudnovsky, convergem com maior rapidez, exigindo menor quantidade de iterações para atingir a mesma tolerância de erro.

De modo análogo, a distribuição das etapas de cálculo entre múltiplos núcleos de processamento (CPUs) ou aceleradores gráficos (GPUs) viabiliza a redução substancial do tempo de execução sem prejuízo da exatidão requerida. Trata-se de uma abordagem indispensável em simulações físicas de larga escala e no treinamento de modelos modernos de inteligência artificial, cenários em que o processamento intensivo em paralelo viabiliza o tratamento de grandes volumes de dados em tempo hábil.

## 6 Conclusão

O experimento demonstra, em escala controlada, um princípio fundamental da computação científica: a obtenção de maior precisão numérica acarreta custo computacional progressivo, o qual cresce de forma desproporcional ao ganho incremental de exatidão na série de Leibniz. Esse princípio orienta decisões de projeto em sistemas de alto desempenho, nos quais a escolha criteriosa de algoritmos eficientes, a aplicação de técnicas de paralelismo e a definição de tolerâncias de erro determinam a viabilidade prática das soluções computacionais.